

HW9 — Distributed Databases using Replication

Architecture Overview

This project implements two distributed Key-Value store architectures:

1. **Leader-Follower** (1 Leader + 4 Followers, N=5) with configurable W/R quorums
2. **Leaderless** (5 equal nodes, W=N, R=1)

Both are in-memory hash-table-as-a-service implementations with logical versioning, artificial delays to simulate real-world latency, and a test suite that demonstrates consistency/inconsistency windows.

Project Structure

```
hw9/
├── leader-follower/
│   ├── main.go          # Leader-Follower KV node (single binary, role via
env)
│   ├── Dockerfile
│   └── docker-compose.yml # 1 leader + 4 followers
├── leaderless/
│   ├── main.go          # Leaderless KV node
│   ├── Dockerfile
│   └── docker-compose.yml # 5 equal nodes
├── loadtester/
│   └── main.go          # Load testing client
├── tests/
│   └── consistency_test.go # Consistency unit tests
├── scripts/
│   ├── run_load_tests.sh # Automated test runner
│   └── generate_graphs.py # Graph generation from CSV results
├── go.mod
└── README.md
```

How It Works

Leader-Follower

A single Go binary runs as either a **leader** or **follower** based on the `NODE_ROLE` environment variable.

Write path (**POST /set**):

1. The leader receives the write, assigns the next logical version, stores locally.
2. The leader sequentially replicates to each follower. After sending each replicate message, the leader **sleeps 200ms**. The follower **sleeps 100ms** before storing and acknowledging.
3. For W=5: the leader waits for all 4 follower acks before responding 201.

4. For $W=1$: the leader responds 201 immediately and replicates asynchronously.
5. For $W=3$: the leader waits for 2 follower acks (leader + 2 followers = 3), responds 201, then continues async replication to the remaining followers.

Read path (GET /get):

1. For $R=1$: return the leader's local value.
2. For $R=3$: query 2 followers in parallel (each sleeps 50ms), compare versions with local, return the highest version.
3. For $R=5$: query all 4 followers in parallel, return the highest version.

Sneaky test endpoint (GET /local_read): Returns the value stored locally on that specific node, bypassing any quorum logic. Used to observe the inconsistency window during testing.

Dynamic configuration (POST /configure): Change W and R at runtime without restarting nodes.

Leaderless

All five nodes are identical. Any node can handle any request.

Write path (POST /set):

1. The receiving node becomes the **Write Coordinator** for that request.
2. It stores locally, then sequentially replicates to all 4 peers (200ms sleep after each, peers sleep 100ms).
3. Since $W=N$, it waits for ALL peers to acknowledge before returning 201 to the client.

Read path (GET /get):

1. $R=1$, so the node simply returns its own local value. This creates an inconsistency window: if a write is in-progress on another node, this node may return stale data.

Artificial Delays (Simulating Real Storage)

Event	Delay
Leader/Coordinator sleeps after each follower/peer message	200ms
Follower/Peer processes incoming replicate	100ms
Follower responds to leader's read request	50ms

These delays make the inconsistency window large enough to observe and test.

Version Numbers

Each KV entry carries a monotonically increasing logical version. On write, the leader/coordinator increments the version. On replicate, followers only update if the incoming version is strictly higher than what they have. The load tester tracks versions client-side to detect stale reads.

How to Run

Prerequisites

- Docker & Docker Compose
- Go 1.21+
- Python 3 with `pandas` and `matplotlib` (for graphs)

Step 1: Start the Leader-Follower Cluster

```
cd leader-follower
docker compose up --build -d
```

This starts 5 containers (leader on port 8080, followers on 8081-8084).

Step 2: Start the Leaderless Cluster

```
cd ../leaderless
docker compose up --build -d
```

This starts 5 containers (nodes on ports 9080-9084).

Step 3: Verify Health

```
curl http://localhost:8080/health
curl http://localhost:9080/health
```

Step 4: Run Consistency Tests

```
cd .. # back to hw9 root
go test -v ./tests/
```

Step 5: Run Load Tests

```
bash scripts/run_load_tests.sh
```

This runs all 16 load test combinations (4 configs × 4 write ratios) and saves CSVs to `results/`.

Step 6: Generate Graphs

```
pip install pandas matplotlib
python3 scripts/generate_graphs.py results/*.csv
```

Graphs are saved to `graphs/`.

Step 7: Tear Down

```
cd leader-follower && docker compose down
cd ../leaderless && docker compose down
```

API Reference

Endpoint	Method	Description
<code>/set</code>	POST	Write a key-value pair. Body: <code>{"key":"k","value":"v"}</code> . Returns 201.
<code>/get?key=k</code>	GET	Read a key. Returns 200 + value or 404.
<code>/local_read?key=k</code>	GET	Read local node value only (test endpoint).
<code>/replicate</code>	POST	Internal: receive replicated data from leader/coordinator.
<code>/read_request?key=k</code>	GET	Internal: leader queries follower for quorum reads.
<code>/configure</code>	POST	Set W and R dynamically. Body: <code>{"w":3,"r":3}</code> .
<code>/health</code>	GET	Health check returning node ID and role.

Load Tester Design

The load tester uses a **small key space** (default: 10 keys) to ensure temporal locality — reads and writes to the same key cluster closely in time. This is the simplest effective strategy: with only 10 keys and concurrent workers, the probability of a read hitting a recently-written key is high.

Each worker randomly decides read vs. write based on the configured ratio, picks a random key from the small pool, and fires the request. The client tracks the latest version written per key, so when a read returns, it can detect staleness by comparing versions.

Output per request: type, key, latency (ms), HTTP status, version, stale flag, timestamp, and the time interval since the last write to that same key.

Error Handling & Edge Cases

- **Network failures:** HTTP client has a 5-second timeout. Failed replication attempts are logged but don't crash the node.
- **Version conflicts:** Followers only accept updates with a strictly higher version number, preventing stale overwrites.
- **Empty keys:** Rejected with 400 Bad Request.
- **Missing keys:** GET returns 404.

- **Concurrent writes:** The leader's version assignment is protected by a mutex. In leaderless mode, an atomic counter ensures unique versions per node.
-